# How to run the code

Assignment 1: Reinforcement Learning · Vincent Yang · https://github.com/vi-yang-NU/RL

These instructions are for grading. Every answer, plot, evaluation statistic and validation result is in the report, and the repository includes the results the report was built from (`results/`), so **nothing has to be run to grade**. The steps below set up the code to rerun any part, and open a dashboard that shows each part's answers next to its plots and test evidence.

## 1. Quick start

You need Python 3.9 or newer on macOS or Linux (on Windows, use WSL). In a terminal:

```
git clone https://github.com/vi-yang-NU/RL.git
cd RL
./setup.sh --no-ssh           # creates .venv, installs the packages, checks the environment
source .venv/bin/activate
python run.py ui              # opens the dashboard in your browser at http://localhost:8501
```

`setup.sh` installs the packages in `requirements.txt` (NumPy, Matplotlib, PyTorch, Gymnasium with the Atari environments, ale-py) and `requirements-ui.txt` (Streamlit, pandas, reportlab) into a virtual environment in `.venv`; the first time takes a few minutes because of PyTorch. It ends with `python run.py check`, whose last line should read `ready`. `--no-ssh` skips the optional SSH setup for Northwestern's deepdish servers. Stop the dashboard with Ctrl+C in the terminal.

Without the script:

```
python3 -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt -r requirements-ui.txt
python run.py check
```

## 2. Where to find each answer

The report and the dashboard follow the handout's order and use the same headings. In the dashboard, open the tab, then scroll to the section; in the report, the same section is under the part's heading.

| Handout item | Dashboard tab › section | Rerun with |
|---|---|---|
| GitHub username, repository and link; these instructions | Overview | |
| 2: MDP formulation and assumptions | 2 · Value & Policy Iteration › MDP formulation, Assumptions | |
| 2 (a) Enumeration, T = 500: optimal value function at time 0 | 2 · Value & Policy Iteration › (a) Enumeration, T = 500 | `python run.py dp --method enumeration` |
| 2 (b) Value iteration: optimal value function | 2 · Value & Policy Iteration › (b) Value iteration, T = ∞ | `python run.py dp --method value_iteration` |

| Handout item | Dashboard tab › section | Rerun with |
|---|---|---|
| 2 (c) Policy iteration: optimal policy | 2 · Value & Policy Iteration › (c) Policy iteration | `python run.py dp --method policy_iteration` |
| 2: edge cases, validation tests, 3 more months | 2 · Value & Policy Iteration › Edge cases, Tests for validating..., Additional validation... | `python run.py tests --section dp` |
| Part 1 (a) CartPole-v1: training curve, histogram, mean and standard deviation | 3 · Policy Gradient Part 1 › (a) CartPole-v1 | `python run.py pg --part 1 --env cartpole` |
| Part 1 (b) Pong-v5: the same | 3 · Policy Gradient Part 1 › (b) Pong-v5 | `python run.py pg --part 1 --env pong` |
| Part 1: edge cases, validation tests, 3 more months | 3 · Policy Gradient Part 1 › Edge cases, Tests for validating..., Additional validation... | `python run.py tests --section pg1` |
| Part 2: the baseline chosen and how it is implemented | Policy Gradient Part 2 › Baseline: choice and implementation | |
| Part 2 (a) CartPole-v1 with the baseline | Policy Gradient Part 2 › (a) CartPole-v1 with baseline | `python run.py pg --part 2 --env cartpole` |
| Part 2 (b) Pong-v5 with the baseline | Policy Gradient Part 2 › (b) Pong-v5 with baseline | `python run.py pg --part 2 --env pong` |
| Part 2: edge cases, validation tests, 3 more months | Policy Gradient Part 2 › Edge cases, Tests for validating..., Additional validation... | `python run.py tests --section pg2` |

The files behind them are in `results/` (section 8 lists the folders).

## 3. Using the dashboard

`python run.py ui` opens the dashboard at http://localhost:8501.

- **Tabs** along the top:
  - **Overview**: the submission details, a table of where each part is, progress, and the report and these instructions as PDFs to download.
  - **2 · Value & Policy Iteration**, **3 · Policy Gradient Part 1** and **Policy Gradient Part 2**: one tab per part of the handout.
- **Contents**, at the right edge of the page, lists the sections of the open tab. The entry for the section on screen is highlighted, and the highlight glides to the next entry as you scroll; click an entry to jump to it. (It appears when the window is at least about 1250 px wide.)
- **A part's tab** lists the handout's questions in order, each in its own box: the question quoted from the handout, the required plots, for policy-gradient runs the mean and standard deviation over the 500 evaluation episodes, the validation tests, and the **Answer**. Click a validation test to expand it: what it does, how it validates the algorithm, each check (✓ or ✗), and its metrics, tables and figures.
- **Explore the problem** (tab 2, under the Run panel) is an interactive view of Section 2. Change any constant or structural switch of the shuttle problem on the left (or pick a preset

such as deterministic arrivals, γ = 0, a tiny station or two shuttles per period) and the three solvers rerun immediately. The page on the right shows the two stations with the waiting and delivered customers; **Step**, **Play** and **Back** replay the first 10 periods of the chosen policy, the action buttons take a different action in a period, and the tiles and the period table show the state, the cost and its breakdown, V*(s_t), Q*(s_t, ·) and the running discounted total at each step. The panel beside it steps through the first 10 iterations of enumeration, value iteration or policy iteration (values at selected states, the dispatch threshold, the change per step), and the charts show V*, the policy bands and the convergence for the selected step. **Run this case in the Run panel** (under the widgets) copies the current parameters into the Run panel at the top of the tab, as `--set` arguments with a run tag so the handout's results stay, and scrolls up to it; press **Launch** there.

- **The Run panel** at the top of each part's tab reruns that part; click its header to collapse it. Choose a job (for example *(a) CartPole-v1* or *Validation tests*), optionally add a run tag and extra arguments, and press **Launch**. Under **Jobs**, choose a job to see its log and, while a policy-gradient run trains, its reward curve (turn on auto-refresh to follow it). When the job has finished, press R to reload the page and its plots.
- **The sidebar** on the left sets where jobs run: **this computer** (the default) or a deepdish server, and the GPU. Hide it with « at its top; bring it back with » at the top left of the page. The deepdish connection settings and the **Set up deepdish** panel in it are only needed to run on deepdish.

Rerunning a part overwrites its results in your copy of the repository; `git checkout results/` restores the submitted ones. For a policy-gradient run, a **Run tag** (or `--tag NAME` on the command line) writes to a separate folder instead and leaves the submitted run in place.

## If the dashboard does not load

Nothing in the dashboard is needed to grade: the report (`report/report.pdf`) has every answer, plot, statistic and test result, and the files behind them are in `results/` (section 8). If the page stays blank, keeps saying "Please wait" or shows a connection error:

1. Wait for the terminal to print `Dashboard: http://localhost:8501`: the first start takes up to a minute because of PyTorch. Then reload the page (press R in the page, or the browser's reload button).
2. If no browser opened, or it opened before the dashboard was ready, open http://localhost:8501 yourself. Over SSH the dashboard does not open a browser: forward the port (`ssh -L 8501:localhost:8501 …`) and open that address on your computer.
3. If the port is in use (the terminal says so), start it elsewhere: `python run.py ui --port 8502`.
4. If `python run.py ui` fails to start, install the dashboard's packages with `pip install -r requirements-ui.txt` (with `.venv` active), or run Streamlit directly: `python -m streamlit run ui/app.py`.
5. Try another browser or a private window: some extensions and strict ad blockers block the page's WebSocket connection, which is what the dashboard uses to draw.

If it still does not load, grade from the report: section 2 names the report section for each handout item, section 4 reruns any part from the command line, and `python run.py report` rebuilds the report from `results/`.

## 4. Rerunning from the command line

Every part runs through one entry point, `run.py`, from the repository root with the virtual environment active (`source .venv/bin/activate`); `python run.py -h` lists the commands.

| Part | Command | Results in |
|---|---|---|
| Section 2 (a)–(c): enumeration, value and policy iteration | `python run.py dp` | `results/dp/` |
| Part 1 (a): CartPole-v1 | `python run.py pg --part 1 --env cartpole` | `results/pg/part1_cartpole/` |
| Part 1 (b): Pong-v5 | `python run.py pg --part 1 --env pong` | `results/pg/part1_pong/` |
| Part 2 (a): CartPole-v1 with a baseline | `python run.py pg --part 2 --env cartpole` | `results/pg/part2_cartpole/` |
| Part 2 (b): Pong-v5 with a baseline | `python run.py pg --part 2 --env pong` | `results/pg/part2_pong/` |

**Section 2.** `python run.py dp` solves the shuttle-dispatch MDP (K = 15, c_f = 100, c_h = 2, A_t ~ Unif{1, ..., 5}, at most 200 customers waiting, γ = 0.95) with all three methods; `--method enumeration`, `--method value_iteration` or `--method policy_iteration` runs one of them. Each method saves its solution as JSON and its plot: `enumeration_value.png`, `value_iteration_value.png` and `policy_iteration_policy.png`, plus a convergence plot. Any constant of the problem can be changed with `--set`, for example `python run.py dp --set capacity=10 --set gamma=0.9 --set 'arrivals=[1,2,3]'` (`rl/dp/shuttle.py` lists the fields, including the structural switches `max_dispatch` and `holding_charged`). `--backend cuda` runs the same three methods with the CUDA kernels in `rl/cuda/dp.py` (one thread per state, the value function in shared memory, the convergence loop inside the kernel); on a machine without an NVIDIA GPU they run in Numba's CPU simulator, which is slow but gives the same numbers. `--explorer results/dp/explore.html` also writes the dashboard's interactive explorer (section 3) for the chosen parameters as a standalone page that opens in any browser.

**Policy gradient.** Each run trains the policy network with Adam, then rolls out 500 episodes with the trained policy. CartPole-v1 trains on a CPU; Pong-v5 is much slower and is best run on a GPU (section 6). A run's folder contains:

- `training_curve.png`: episode reward against training episode, with the 100-episode moving average;
- `eval_histogram.png`: histogram of the 500 evaluation rewards;
- `eval.json`: the 500 evaluation rewards, their mean and their standard deviation;
- `config.json` (the settings used) and `train.json`.

The defaults follow the handout: learning rate 0.01, discount factor 0.95 for CartPole and 0.99 for Pong, and 500 evaluation episodes. An episode ends when the environment ends it (CartPole-v1 stops itself after 500 steps; a Pong game ends at 21 points) or after an upper limit of 1000 steps (`--set max_steps=N`; for Pong the limit is 10000, because a game lasts 1000-3000 steps and a 1000-step cap would cut games short). Options:

| Option | Effect |
|---|---|
| `--episodes N` | number of training episodes (default 1000 for CartPole, 2000 for Pong) |
| `--lr X`, `--gamma X` | Adam learning rate and discount factor |
| `--baseline NAME` | Part 2 baseline: `value` (learned state-value network, the default), `mean` (constant moving average) or `time` (per time step) |
| `--seed N` | random seed (default 0) |
| `--device D` | `auto` (the GPU when one is visible, else the CPU), `cpu`, `cuda`, `cuda:N` or `mps` (Apple GPU); `cuda` and `mps` fail if unavailable. The policy, the baseline network and every tensor they use are placed on that device; the run prints the GPU it uses |
| `--eval-episodes N` | number of evaluation episodes (default 500) |
| `--tag NAME` | save to `results/pg/<run>_NAME/`, next to the submitted run |
| `--eval-only` | skip training and evaluate the run's saved `policy.pt` |
| `--set KEY=VALUE` | any tunable in `rl/pg/options.py` (repeatable): network sizes (`'hidden=[128,64]'`), `arch=cnn` for Pong, `batch_episodes`, `normalize`, `reward_to_go`, `discount_grad`, `entropy_coef`, `max_grad_norm`, `eval_greedy`, the baseline's learning rate, `kernels=true` (discounted returns and Pong preprocessing with the CUDA kernels in `rl/cuda/pg.py`), … |

## 5. Validation experiments

The experiments that check that each algorithm behaves as predicted are in `tests/test_dp.py`, `tests/test_pg_part1.py` and `tests/test_pg_part2.py`. Each one states what it does and how it validates the algorithm; the report's "Tests for validating the correctness of your algorithm" sections list them with their results.

```
python run.py tests --list                # every experiment, by section
python run.py tests --section dp          # dp, pg1, pg2 or all
python run.py tests --section pg1 -k TEXT # only experiments whose name or title contains TEXT
```

Each experiment ends as **passed** (all its checks hold), **failed** (a check does not hold), **error** (it raised an exception) or **recorded** (evidence without pass/fail checks). Results go to `results/tests/<section>.json`, with figures in `results/tests/<section>/`. All of them run on a laptop CPU in about two minutes.

**Unit tests.** `python run.py unittest` runs about 110 unit tests (`tests/unit/`, standard-library `unittest`) of the building blocks and edge cases: MDP construction, the solvers on problems with closed-form solutions, exact-arithmetic references, discounted returns, policy networks, environment adapters, training on toy environments, the baselines,

device placement, and the CUDA kernels (compared with the CPU code; on a machine without an NVIDIA GPU they run in Numba's simulator, which adds about a minute).

**Lean proofs.** `lean/` contains Lean 4 (Mathlib) proofs of the properties the tests check against: the Bellman operator of the shuttle MDP is a γ-contraction and the value-iteration error bound is sound, the optimal value is monotone in the horizon and in the queue, policy improvement never makes a policy worse, the backward recursion for discounted returns equals its definition, and an action-independent baseline leaves the expected policy gradient unchanged. The exact reference values computed by the verified Lean definition are committed in `lean/reference/shuttle_reference.json` and compared with the Python solvers by the tests, so Lean does not have to be installed to grade; `lean/README.md` explains how to rebuild the proofs (`lake build`) if you wish.

## 6. Running on deepdish (one GPU card)

The handout requires a single GPU card on deepdish. Clone the repository there, run `./setup.sh --server` (it detects the servers' GTX 1080 cards and installs a PyTorch build that supports them, and puts the environment on the local disk because importing PyTorch from the NFS home takes minutes), check which cards are free with `nvidia-smi`, and choose one with `CUDA_VISIBLE_DEVICES`:

```
ssh <netid>@mlds-deepdish3.ads.northwestern.edu       # campus network or the VPN
cd RL && source .venv/bin/activate
CUDA_VISIBLE_DEVICES=0 python run.py pg --part 1 --env pong
```

For long runs, `scripts/jobs.sh` starts the command in the background on one card, so it keeps running after you log out:

```
# arguments: job name, GPU index, command
scripts/jobs.sh start part1_pong 0 python run.py pg --part 1 --env pong
scripts/jobs.sh status                     # state, exit code and GPU of every job
scripts/jobs.sh tail part1_pong 50         # last 50 lines of the job's log
scripts/jobs.sh stop part1_pong
```

## 7. Rebuilding the report

```
python run.py report
```

writes `report/report.md` and `report/report.pdf` (the report) and `report/instructions.pdf` (these instructions) from the saved answers, plots, evaluation statistics and test results. The **Overview** tab of the dashboard does the same.

## 8. Repository layout

| Path | Contents |
|---|---|
| `run.py` | entry point for every command above |
| `rl/dp/` | Section 2: the shuttle-dispatch MDP, enumeration, value iteration and policy iteration |
| `rl/pg/` | policy gradient: environments, policy network, training and evaluation, baselines |
| `rl/cuda/` | CUDA kernels (Numba) for Section 2 and the policy-gradient arithmetic, with a CPU simulator fallback |
| `rl/plotting.py` | the plots the handout requires |
| `tests/` | the validation experiments and the harness that runs them; `tests/unit/` the unit tests |
| `lean/` | Lean 4 proofs about the algorithms and the exact reference values they produce |
| `results/dp/`, `results/pg/<run>/`, `results/tests/` | Section 2 results, one folder per policy-gradient run, validation results |
| `report/` | the report (`report.pdf`, `report.md`), the saved answers, these instructions and the report builder |
| `ui/` | the dashboard |
| `scripts/jobs.sh` | background job runner, one GPU card per job |